Guía de Estudio – Estructuras de Datos y Simulación (STRUCT)

Programación Básica · Evidencias de código y corridas
Alumno: _____ · Fecha de ejecución: 2026-10-02 · Python 3.11.15

Índice de programas

1. Sección 2 – Listas (operaciones, recorrido, registro de compras) – 01_listas.py
2. Sección 3 – Tuplas – 02_tuplas.py
3. Sección 4 – Conjuntos – 03_conjuntos.py
4. Sección 5 – Diccionarios – 04_diccionarios.py
5. Sección 6 – Estructuras combinadas – 05_estructuras_combinadas.py
6. Ejercicio 1 – Análisis de encuestas – 06_ejercicio1_encuestas.py
7. Ejercicio 2 – Agenda digital – 07_ejercicio2_agenda.py
8. Ejercicio 3 – Control de inventario – 08_ejercicio3_inventario.py
9. Struct 1 – Clases – 09_struct_clase.py
10. Struct 2 – namedtuple – 10_struct_namedtuple.py
11. Struct 3 – dataclasses – 11_struct_dataclass.py
12. Struct 4 – módulo struct (binario) – 12_struct_modulo.py
13. Ejemplo práctico combinado (veterinaria) – 13_ejemplo_combinado.py
14. Ejercicio integrador final – Veterinaria – 14_integrador_veterinaria.py

Sección 2 – Listas (operaciones, recorrido, registro de compras)

Archivo: **01_listas.py** · código de salida: 0

Código

```
# Seccion 2 - Listas (list)
frutas = ["manzana", "pera", "uva"]

print(frutas[0])            # Acceso: "manzana"
frutas.append("kiwi")       # Agregar
frutas.remove("pera")       # Eliminar
frutas[1] = "plátano"       # Actualizar
print(frutas)

# Recorrido
for fruta in frutas:
    print(fruta)

# Ejemplo practico - Registro de compras
compras = []
for i in range(3):
    producto = input("Producto: ")
    compras.append(producto)

print("Lista final:", compras)
```

Corrida

```
[Entradas tecleadas: leche, pan, huevos]

manzana
['manzana', 'plátano', 'kiwi']
manzana
plátano
kiwi
Producto: Producto: Producto: Lista final: ['leche', 'pan', 'huevos']
```

Sección 3 – Tuplas

Archivo: **02_tuplas.py** · código de salida: 0

Código

```
# Seccion 3 - Tuplas (tuple)
coordenadas = (35.67, -120.45)
print("Coordenadas:", coordenadas)

# Ejemplo - Puntos geograficos
ciudad = ("León", 21.12, -101.68)
print(f"Ciudad: {ciudad[0]}, Latitud: {ciudad[1]}")

# Inmutabilidad: intentar modificar produce error
try:
    ciudad[1] = 0
except TypeError as e:
    print("Error esperado (tupla inmutable):", e)
```

Corrida

```
Coordenadas: (35.67, -120.45)
Ciudad: León, Latitud: 21.12
Error esperado (tupla inmutable): 'tuple' object does not support item assignment
```

Sección 4 – Conjuntos

Archivo: **03_conjuntos.py** · código de salida: 0

Código

```
# Seccion 4 - Conjuntos (set)
materias = {"Matemáticas", "Física", "Programación"}

# Operaciones
materias.add("Inglés")
materias.discard("Física")
print("Programación" in materias)
print(sorted(materias))

# Operaciones matematicas
a = {"Python", "C++", "Java"}
b = {"Python", "JavaScript"}
print(sorted(a.union(b)))         # Union
print(sorted(a.intersection(b)))  # Interseccion
```

Corrida

```
True
['Inglés', 'Matemáticas', 'Programación']
['C++', 'Java', 'JavaScript', 'Python']
['Python']
```

Sección 5 – Diccionarios

Archivo: **04_diccionarios.py** · código de salida: 0

Código

```
# Seccion 5 - Diccionarios (dict)
alumno = {"nombre": "Ana", "edad": 20, "carrera": "Sistemas"}

print(alumno["nombre"])
alumno["edad"] = 21         # Actualizar
alumno["promedio"] = 9.2    # Agregar
del alumno["carrera"]       # Eliminar

# Recorrido
for clave, valor in alumno.items():
    print(clave, ":", valor)
```

Corrida

```
Ana
nombre : Ana
edad : 21
promedio : 9.2
```

Sección 6 – Estructuras combinadas

Archivo: **05_estructuras_combinadas.py** · código de salida: 0

Código

```
# Seccion 6 - Estructuras combinadas: registro de estudiantes
estudiantes = [
    {"nombre": "Ana", "edad": 20, "promedio": 9.1},
    {"nombre": "Luis", "edad": 22, "promedio": 8.5},
    {"nombre": "María", "edad": 21, "promedio": 9.3},
]

for est in estudiantes:
    print(f"{est['nombre']} tiene promedio {est['promedio']}")
```

Corrida

```
Ana tiene promedio 9.1
Luis tiene promedio 8.5
María tiene promedio 9.3
```

Ejercicio 1 – Análisis de encuestas

Archivo: **06_ejercicio1_encuestas.py** · código de salida: 0

Código

```
# Ejercicio 1 - Analisis de encuestas
lenguajes = ["Python", "C++", "Python", "Java", "Python", "C#"]

# Cuantos estudiantes dominan cada lenguaje (sin duplicados)
unicos = set(lenguajes)
for l in sorted(unicos):
    print(l, "→", lenguajes.count(l))

print("Lenguajes sin duplicados:", sorted(unicos))
```

Corrida

```
C# → 1
C++ → 1
Java → 1
Python → 3
Lenguajes sin duplicados: ['C#', 'C++', 'Java', 'Python']
```

Ejercicio 2 – Agenda digital

Archivo: **07_ejercicio2_agenda.py** · código de salida: 0

Código

```
# Ejercicio 2 - Agenda digital (agregar, buscar, eliminar, listar)
agenda = {"Ana": "4771234567", "Luis": "4778901234"}


def agregar():
    nombre = input("Nombre a agregar: ")
    agenda[nombre] = input("Teléfono: ")
    print(f"Contacto {nombre} agregado.")


def buscar():
    nombre = input("Nombre a buscar: ")
    if nombre in agenda:
        print(f"Teléfono: {agenda[nombre]}")
    else:
        print("No encontrado")


def eliminar():
    nombre = input("Nombre a eliminar: ")
    if nombre in agenda:
        del agenda[nombre]
        print(f"Contacto {nombre} eliminado.")
    else:
        print("No encontrado")


def listar():
    for nombre, tel in agenda.items():
        print(f"{nombre}: {tel}")


acciones = {"1": agregar, "2": buscar, "3": eliminar, "4": listar}

while True:
    print("\n--- AGENDA ---\n1) Agregar  2) Buscar  3) Eliminar  4) Listar  5) Salir")
    op = input("Opción: ")
    if op == "5":
        break
    acciones.get(op, lambda: print("Opción inválida"))()
```

Corrida

```
[Entradas tecleadas: 2, Ana, 1, Pedro, 5551112222, 2, Pedro, 3, Luis, 2, Luis, 4, 5]


--- AGENDA ---
1) Agregar  2) Buscar  3) Eliminar  4) Listar  5) Salir
Opción: Nombre a buscar: Teléfono: 4771234567

--- AGENDA ---
1) Agregar  2) Buscar  3) Eliminar  4) Listar  5) Salir
Opción: Nombre a agregar: Teléfono: Contacto Pedro agregado.

--- AGENDA ---
1) Agregar  2) Buscar  3) Eliminar  4) Listar  5) Salir
Opción: Nombre a buscar: Teléfono: 5551112222

--- AGENDA ---
1) Agregar  2) Buscar  3) Eliminar  4) Listar  5) Salir
Opción: Nombre a eliminar: Contacto Luis eliminado.

--- AGENDA ---
1) Agregar  2) Buscar  3) Eliminar  4) Listar  5) Salir
Opción: Nombre a buscar: No encontrado

--- AGENDA ---
1) Agregar  2) Buscar  3) Eliminar  4) Listar  5) Salir
Opción: Ana: 4771234567
Pedro: 5551112222

--- AGENDA ---
1) Agregar  2) Buscar  3) Eliminar  4) Listar  5) Salir
Opción:
```

Ejercicio 3 – Control de inventario

Archivo: **08_ejercicio3_inventario.py** · código de salida: 0

Código

```
# Ejercicio 3 - Control de inventario
inventario = [
    {"producto": "Croquetas", "precio": 250, "stock": 20},
    {"producto": "Juguete", "precio": 100, "stock": 15},
    {"producto": "Shampoo", "precio": 180, "stock": 10},
]

# Calcular ingresos potenciales
total = 0
for item in inventario:
    subtotal = item["precio"] * item["stock"]
    print(f"{item['producto']}: {item['precio']} x {item['stock']} = {subtotal}")
    total += subtotal

print("Ingreso total si se vende todo:", total)
```

Corrida

```
Croquetas: 250 x 20 = 5000
Juguete: 100 x 15 = 1500
Shampoo: 180 x 10 = 1800
Ingreso total si se vende todo: 8300
```

Struct 1 – Clases

Archivo: **09_struct_clase.py** · código de salida: 0

Código

```
# Struct en Python (1) - Usando clases
class Mascota:
    def __init__(self, nombre, especie, edad, vacuna):
        self.nombre = nombre
        self.especie = especie
        self.edad = edad
        self.vacuna = vacuna


m1 = Mascota("Firulais", "Perro", 3, "Rabia")
m2 = Mascota("Michi", "Gato", 2, "Triple Felina")

print(m1.nombre, "-", m1.vacuna)
print(m2.nombre, "-", m2.vacuna)
```

Corrida

```
Firulais - Rabia
Michi - Triple Felina
```

Struct 2 – namedtuple

Archivo: **10_struct_namedtuple.py** · código de salida: 0

Código

```
# Struct en Python (2) - namedtuple
from collections import namedtuple

Mascota = namedtuple("Mascota", ["nombre", "especie", "edad", "vacuna"])

m1 = Mascota("Firulais", "Perro", 3, "Rabia")
m2 = Mascota("Michi", "Gato", 2, "Triple Felina")

print(m1.nombre)   # Acceso por nombre
print(m2.vacuna)

try:
    m1.edad = 4
except AttributeError as e:
    print("Error esperado (namedtuple inmutable):", e)
```

Corrida

```
Firulais
Triple Felina
Error esperado (namedtuple inmutable): can't set attribute
```

Struct 3 – dataclasses

Archivo: **11_struct_dataclass.py** · código de salida: 0

Código

```
# Struct en Python (3) - dataclasses
from dataclasses import dataclass


@dataclass
class Mascota:
    nombre: str
    especie: str
    edad: int
    vacuna: str


m1 = Mascota("Firulais", "Perro", 3, "Rabia")
m2 = Mascota("Michi", "Gato", 2, "Triple Felina")

print(m1)
print(m2.nombre)
```

Corrida

```
Mascota(nombre='Firulais', especie='Perro', edad=3, vacuna='Rabia')
Michi
```

Struct 4 – módulo struct (binario)

Archivo: **12_struct_modulo.py** · código de salida: 0

Código

```
# Struct en Python (4) - modulo struct (manejo binario)
import struct

# Empaquetar datos (entero y flotante) en formato binario
data = struct.pack('if', 25, 3.14)   # i = int, f = float
print("Datos empaquetados:", data)

# Desempaquetar los datos
valor = struct.unpack('if', data)
print("Datos originales:", valor)
```

Corrida

```
Datos empaquetados: b'\x19\x00\x00\x00\xc3\xf5H@'
Datos originales: (25, 3.140000104904175)
```

Ejemplo práctico combinado (veterinaria)

Archivo: **13_ejemplo_combinado.py** · código de salida: 0

Código

```
# Ejemplo practico combinado - Veterinaria con dataclass
from dataclasses import dataclass


@dataclass
class Mascota:
    nombre: str
    especie: str
    edad: int
    vacuna: str
    costo: float


registro = [
    Mascota("Firulais", "Perro", 3, "Rabia", 250.0),
    Mascota("Michi", "Gato", 2, "Triple Felina", 300.0),
    Mascota("Max", "Perro", 1, "Parvovirus", 220.0),
]

total = 0
for m in registro:
    print(f"{m.nombre} ({m.especie}) - Vacuna: {m.vacuna} - ${m.costo}")
    total += m.costo
print("\nTotal ingresos:", total)
```

Corrida

```
Firulais (Perro) - Vacuna: Rabia - $250.0
Michi (Gato) - Vacuna: Triple Felina - $300.0
Max (Perro) - Vacuna: Parvovirus - $220.0

Total ingresos: 770.0
```

Ejercicio integrador final – Veterinaria

Archivo: **14_integrador_veterinaria.py** · código de salida: 0

Código

```
# Ejercicio integrador final - Gestion de una veterinaria
# Usa listas y diccionarios combinados.
PRECIOS = {"Rabia": 250.0, "Triple Felina": 300.0, "Parvovirus": 220.0, "Moquillo": 180.0}

mascotas = []  # lista de diccionarios


def registrar():
    nombre = input("Nombre: ").strip()
    tipo = input("Tipo (perro/gato/...): ").strip()
    edad = int(input("Edad: "))
    print("Vacunas disponibles:", ", ".join(f"{v} (${p})" for v, p in PRECIOS.items()))
    vacuna = input("Vacuna aplicada: ").strip()
    if vacuna not in PRECIOS:
        print("Vacuna no válida, no se registró.")
        return
    mascotas.append({"nombre": nombre, "tipo": tipo, "edad": edad, "vacuna": vacuna})
    print(f"Mascota {nombre} registrada.")


def buscar():
    nombre = input("Nombre a buscar: ").strip().lower()
    encontradas = [m for m in mascotas if m["nombre"].lower() == nombre]
    if not encontradas:
        print("No encontrada")
    for m in encontradas:
        print(f"{m['nombre']} - {m['tipo']} - {m['edad']} años - vacuna: {m['vacuna']}")


def estadisticas():
    conteo = {}
    for m in mascotas:
        conteo[m["vacuna"]] = conteo.get(m["vacuna"], 0) + 1
    print("Vacunas aplicadas:")
    for vacuna, n in conteo.items():
        print(f"  {vacuna}: {n}")
    print("Total de vacunas:", sum(conteo.values()))


def ingreso_total():
    desglose = {}
    for m in mascotas:
        desglose[m["vacuna"]] = desglose.get(m["vacuna"], 0) + PRECIOS[m["vacuna"]]
    for vacuna, monto in desglose.items():
        print(f"  {vacuna}: ${monto}")
    print("Ingreso total:", sum(desglose.values()))


acciones = {"1": registrar, "2": buscar, "3": estadisticas, "4": ingreso_total}

while True:
    print("\n--- VETERINARIA ---\n1) Registrar  2) Buscar  3) Estadísticas  4) Ingreso total  5) Salir")
    op = input("Opción: ")
    if op == "5":
        break
    acciones.get(op, lambda: print("Opción inválida"))()
```

Corrida

```
[Entradas tecleadas: 1, Firulais, Perro, 3, Rabia, 1, Michi, Gato, 2, Triple, Felina, 1, Max, Perro, 1, Rabia, 1,
Luna, Gato, 4, Moquillo, 2, michi, 2, Zeus, 3, 4, 5]


--- VETERINARIA ---
1) Registrar  2) Buscar  3) Estadísticas  4) Ingreso total  5) Salir
Opción: Nombre: Tipo (perro/gato/...): Edad: Vacunas disponibles: Rabia ($250.0), Triple Felina ($300.0), Parvovirus
($220.0), Moquillo ($180.0)
Vacuna aplicada: Mascota Firulais registrada.

--- VETERINARIA ---
1) Registrar  2) Buscar  3) Estadísticas  4) Ingreso total  5) Salir
Opción: Nombre: Tipo (perro/gato/...): Edad: Vacunas disponibles: Rabia ($250.0), Triple Felina ($300.0), Parvovirus
($220.0), Moquillo ($180.0)
Vacuna aplicada: Mascota Michi registrada.

--- VETERINARIA ---
```

```
1) Registrar  2) Buscar  3) Estadísticas  4) Ingreso total  5) Salir
Opción: Nombre: Tipo (perro/gato/...): Edad: Vacunas disponibles: Rabia ($250.0), Triple Felina ($300.0), Parvovirus
($220.0), Moquillo ($180.0)
Vacuna aplicada: Mascota Max registrada.

--- VETERINARIA ---
1) Registrar  2) Buscar  3) Estadísticas  4) Ingreso total  5) Salir
Opción: Nombre: Tipo (perro/gato/...): Edad: Vacunas disponibles: Rabia ($250.0), Triple Felina ($300.0), Parvovirus
($220.0), Moquillo ($180.0)
Vacuna aplicada: Mascota Luna registrada.

--- VETERINARIA ---
1) Registrar  2) Buscar  3) Estadísticas  4) Ingreso total  5) Salir
Opción: Nombre a buscar: Michi - Gato - 2 años - vacuna: Triple Felina

--- VETERINARIA ---
1) Registrar  2) Buscar  3) Estadísticas  4) Ingreso total  5) Salir
Opción: Nombre a buscar: No encontrada

--- VETERINARIA ---
1) Registrar  2) Buscar  3) Estadísticas  4) Ingreso total  5) Salir
Opción: Vacunas aplicadas:
  Rabia: 2
  Triple Felina: 1
  Moquillo: 1
Total de vacunas: 4

--- VETERINARIA ---
1) Registrar  2) Buscar  3) Estadísticas  4) Ingreso total  5) Salir
Opción:   Rabia: $500.0
  Triple Felina: $300.0
  Moquillo: $180.0
Ingreso total: 980.0

--- VETERINARIA ---
1) Registrar  2) Buscar  3) Estadísticas  4) Ingreso total  5) Salir
Opción:
```